

House Price Prediction

Author: Onkar Swaroop

Batch: Data Analytics [December 2024]

Email: onkarswaroop98@gmail.com

Introduction

House pricing plays a vital role in the real estate market, influencing the decisions of buyers, sellers, investors and financial institutions. The ability to accurately predict house prices is essential for **property valuation, mortgage assessments and investment planning**. Several factors, including **location, property size, number of rooms, construction quality, and market trends**, impact housing prices. By leveraging machine learning, we can develop a **data-driven model** that analyses these features and predicts property values with high accuracy.

This project focuses on building a **robust house price prediction model** that utilizes **historical housing data** and applies machine learning techniques to identify the key drivers of property prices. The insights gained from this analysis can help **homebuyers make better purchasing decisions, real estate agents price properties competitively, and investors assess market trends for profitable opportunities**.

Project Workflow

The project follows a structured approach to achieve accurate price predictions:

1. **Data Acquisition** – Used the **House Sales in King County dataset**.
 2. **Data Preprocessing** – Handling missing values, encoding categorical features and scaling numerical variables.
 3. **Feature Engineering** – Selecting relevant predictors such as size, location, and condition.
 4. **Model Training & Evaluation** – Used **Linear Regression, Decision Trees, Random Forest and XGBoost**.
 5. **Model Interpretation** – Identified key features affecting house prices through feature importance analysis.
-

Key Concepts Covered

- **Data Preprocessing** – Handling missing values and transforming features.
 - **Machine Learning** – Training models to estimate house prices.
 - **Model Evaluation** – Assessing model performance using RMSE, MAE and R² Score.
 - **Feature Importance** – Identifying key factors influencing property prices.
-

Tech Stack Used

- **Programming Language:** Python
 - **Libraries:** Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, XGBoost
-

Implementation Details

Import necessary libraries

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.ensemble import RandomForestRegressor

from xgboost import XGBRegressor

from sklearn.linear_model import LinearRegression

from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Load Dataset

```
file_path = "/content/kc_house_data.csv"

df = pd.read_csv(file_path)
```

Display first few rows

```
display(df.head())
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	1180	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	2170	40
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	770	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	7	1050	91
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	8	1680	

5 rows × 15 columns

Check dataset info

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     21613 non-null  int64
1   date                   21613 non-null  object
2   price                  21613 non-null  float64
3   bedrooms               21613 non-null  int64
4   bathrooms              21613 non-null  float64
5   sqft_living            21613 non-null  int64
6   sqft_lot               21613 non-null  int64
7   floors                 21613 non-null  float64
8   waterfront             21613 non-null  int64
9   view                   21613 non-null  int64
10  condition              21613 non-null  int64
11  grade                  21613 non-null  int64
12  sqft_above             21613 non-null  int64
13  sqft_basement          21613 non-null  int64
14  yr_built               21613 non-null  int64
15  yr_renovated           21613 non-null  int64
16  zipcode                21613 non-null  int64
17  lat                   21613 non-null  float64
18  long                   21613 non-null  float64
19  sqft_living15          21613 non-null  int64
20  sqft_lot15             21613 non-null  int64
dtypes: float64(5), int64(15), object(1)
memory usage: 3.5+ MB
```

Check for missing values

```
print("\nMissing Values:\n",
df.isnull().sum())
```

```
Missing Values:
id                0
date              0
price             0
bedrooms          0
bathrooms         0
sqft_living       0
sqft_lot          0
floors            0
waterfront        0
view              0
condition         0
grade             0
sqft_above        0
sqft_basement     0
yr_built          0
yr_renovated      0
zipcode           0
lat               0
long              0
sqft_living15     0
sqft_lot15        0
dtype: int64
```

Drop 'id' and 'date' as they are not useful for prediction

```
df.drop(columns=['id', 'date'], inplace=True)
```

Define Features (X) and Target Variable (y)

```
X = df.drop(columns=['price'])
```

```
y = df['price']
```

Split the dataset into training and testing sets (80-20 split)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Feature Scaling (Only for numerical features)

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Initialize models

```
models = {  
    "Linear Regression": LinearRegression(),  
    "Decision Tree": DecisionTreeRegressor(random_state=42),  
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42),  
    "XGBoost": XGBRegressor(n_estimators=100, random_state=42)}
```

Train and evaluate models

```
results = {}  
  
for name, model in models.items():  
    model.fit(X_train_scaled, y_train)  
    y_pred = model.predict(X_test_scaled)
```

Calculate evaluation metrics

```
mae = mean_absolute_error(y_test, y_pred)  
rmse = np.sqrt(mean_squared_error(y_test, y_pred))  
r2 = r2_score(y_test, y_pred)  
  
results[name] = {"MAE": mae, "RMSE": rmse, "R2 Score": r2}  
  
print(f"\n{name} Performance:")  
print(f"Mean Absolute Error (MAE): {mae:.2f}")  
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")  
print(f"R2 Score: {r2:.2f}")
```

Convert results to DataFrame for better visualization

```
results_df = pd.DataFrame(results).T  
display(results_df)
```

Feature Importance (Random Forest)

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)  
rf_model.fit(X_train_scaled, y_train)  
importances = rf_model.feature_importances_  
feature_names = X.columns
```

Linear Regression Performance:
Mean Absolute Error (MAE): 127493.34
Root Mean Squared Error (RMSE): 212539.52
R² Score: 0.70

Decision Tree Performance:
Mean Absolute Error (MAE): 103437.61
Root Mean Squared Error (RMSE): 206863.84
R² Score: 0.72

Random Forest Performance:
Mean Absolute Error (MAE): 72784.74
Root Mean Squared Error (RMSE): 148684.32
R² Score: 0.85

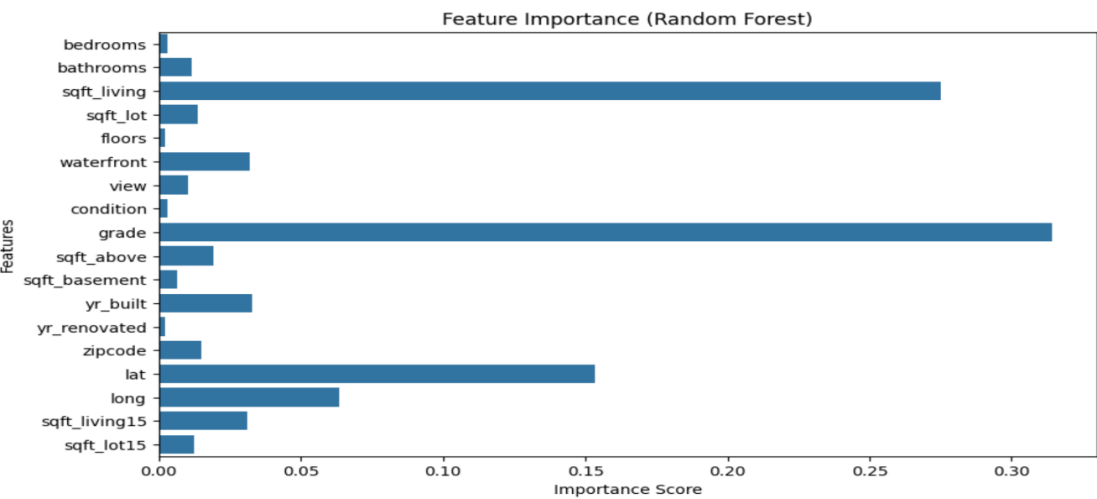
XGBoost Performance:
Mean Absolute Error (MAE): 71751.38
Root Mean Squared Error (RMSE): 147991.93
R² Score: 0.86

	MAE	RMSE	R ² Score
Linear Regression	127493.342087	212539.516638	0.701190
Decision Tree	103437.609877	206863.843898	0.716936
Random Forest	72784.741791	148684.321411	0.853767
XGBoost	71751.380160	147991.934471	0.855126



Plot Feature Importance

```
plt.figure(figsize=(10, 6))  
  
sns.barplot(x=importances, y=feature_names)  
  
plt.xlabel('Importance Score')  
  
plt.ylabel('Features')  
  
plt.title('Feature Importance (Random Forest)')  
  
plt.show()
```



Findings & Insights

Model Performance

Best Model: XGBoost achieved the highest accuracy with $R^2 = 0.86$ and the lowest errors.

Model	MAE	RMSE	R^2 Score
Linear Regression	127493	212539	0.70
Decision Tree	103437	206863	0.72
Random Forest	72784	148684	0.85
XGBoost	71751	147991	0.86

Key Takeaways from Feature Importance

- Larger living space and higher-grade homes have higher prices.
- Waterfront properties significantly increase house value.
- The year built and renovations impact property prices.

Conclusion

This project successfully implemented multiple regression models to predict house prices based on historical data. By analysing key property features such as **square footage, number of bedrooms and bathrooms, location, and renovation history**, the model was able to generate accurate price estimates.

Among the models tested, **XGBoost delivered the best performance**, achieving the highest R^2 score (**0.8551**) and the lowest MAE (\$**71,751**) and RMSE (\$**147,991**). This indicates that **XGBoost can effectively capture complex relationships between features and property prices**, making it a valuable tool for real estate pricing.

The insights derived from feature importance analysis help businesses and buyers make more **data-driven decisions**. Real estate companies can use these predictions to **set competitive pricing strategies**, while homebuyers can better assess whether a property is fairly priced. Investors and financial institutions can also leverage these insights for **mortgage risk assessments and real estate investment planning**.

Moving forward, this model can be **enhanced with additional features**, such as economic indicators, crime rates, and school district ratings, to further improve prediction accuracy. Additionally, integrating this model into a **real estate platform or API** can provide **real-time price estimates** for users, making it a powerful tool for the industry.

Business Implications:

1. **Real Estate Agencies & Sellers:** Can set competitive prices by understanding key factors that drive property value.
 2. **Homebuyers & Investors:** Helps buyers make informed decisions by predicting future property values based on critical features.
 3. **Financial Institutions:** Can improve mortgage risk assessments by leveraging predictive house price models.
-

Scalability & Improvements

While the current model provides strong predictive accuracy, further optimizations can enhance its real-world impact:

- **Exploring Advanced Models** – Further tuning **XGBoost** and **LightGBM** to improve predictive power.
 - **Deploying for Business Use** – Integrating this model into a **real estate platform** or an **API** for real-time property valuation.
 - **Enhancing Interpretability** – Building **interactive dashboards** in **Power BI** or **Tableau** to provide real-time insights to stakeholders.
-

References

Dataset: [House Sales in King County, USA \(Kaggle\)](#)

Scikit-learn Documentation: [Scikit-learn](#)

XGBoost Documentation: [XGBoost](#)

Project Link: [\[Google Colab\]](#) [\[GitHub\]](#) [\[Portfolio\]](#)